

CS T 35- DATA STRUCTURES

UNIT V

Tables: Rectangular tables - Jagged tables – Inverted tables - Symbol tables – Static tree tables - Dynamic tree tables - Hash tables. **Files:** queries - Sequential organization – Index techniques. **External sorting:** External storage devices – Sorting with tapes and disks.

2 Marks

1. Classify external sorting.(Dec 2016)

External sorting is required when the data being sorted do not fit into the main memory of a computing device. And external sorting fall into two types(a).Distribution sorting,(b).External Merge Sort

2. What is hashing?(Dec 2016)

If X is an identifier chosen at random from the identifier space, then we want the probability that $f(X)=i$ to be $1/b$ for all buckets i . then a random has an equal chance of hasting into any of the b buckets. A hash function satisfying this property will be termed a uniform hash function.

Several kinds of uniform hash function are in use.

- i. Mid-square method,
- ii. Division method,
- iii. Folding method and
- iv. Digit analysis method.

3. What is the use of symbol table(Dec 2016)

Symbol table is an important data structure created and maintained by compilers in order to store information about the occurrence of various entities such as variable names, function names, objects, classes, interfaces, etc. **Symbol table** is used by both the analysis and the synthesis parts of a compiler.

4. What is latency Time (Dec 2016).

It is the time required for the disk to rotate to beginning of correct sector.

5. Bring the properties of symbol table (Nov 2015)

Each entry in the table is associated with attributes that support compiler in different phases

6. Write any three External storage devices (Nov 2015, Nov 2017)

- Cache Memory
- Main memory
- Flash memory
- Magnetic disks
- Magnetic tape
- Optical storage

7. Define symbol table(Nov 2017).

Symbol table is a data structure it contains the information about the identifier. Symbol table have two field

Identifier
name
Memory
location

IDENTIFIER NAME MEMORY LOCATION

A	M1
B	M2
C	M3

There are two type of symbol table

- i. Static tree table
- ii. Dynamic tree table

8. Define Jagged table.(May 2018).

Jagged tables are special kind of sparse matrices such as triangular matrices, band matrices etc., In jagged tables, if elements are present then they are contiguous.

9. What is External Sorting (May 2018, Nov 2018).

External sorting is a term for a class of sorting algorithms that can handle massive amounts of data. External sorting is required when the data being sorted do not fit into the main memory of a computing device (usually RAM) and instead they must reside in the slower external memory (usually a hard drive).

10. What is Rehashing (Nov 2018).

Rehashing means hashing again. Basically, when the load factor increases to more than its pre- defined value (default value of load factor is 0.75), the complexity increases. So to overcome this, the size of the array is increased (doubled) and all the values are hashed again and stored in the new double sized array to maintain a low load factor and low complexity.

11. Why Rehashing? What is Index Technique (May 2017)

Rehashing must be done, increasing the size of the bucket Array so as to reduce the load factor and the time complexity. Indexing is a way to optimize the performance of a database by minimizing the number of disk accesses required when a query is processed. It is a data structure technique which is used to quickly locate and access the data in a database.

11 MARKS

1. Describe the different methods available to define the Hash function. (Dec 2014).

Hashing is the process of generating a value from a text or a list of numbers using a mathematical function known as a hash function. There are many hash functions that use numeric numeric or alphanumeric keys. Different hash functions are given below:

Hash Functions

The following are some of the Hash Functions -

Division Method

This is the easiest method to create a hash function. The hash function can be described as -

$$h(k) = k \bmod n$$

Here, $h(k)$ is the hash value obtained by dividing the key value k by size of hash table n using the remainder. It is best that n is a prime number as that makes sure the keys are distributed with more uniformity.

An example of the Division Method is as follows -

$$\begin{aligned} k &= 1276 \\ n &= 10 \\ h(1276) &= 1276 \bmod 10 = 6 \end{aligned}$$

The hash value obtained is 6 disadvantage of the division method is that consecutive keys map to consecutive hash values in the hash table. This leads to a poor performance.

Multiplication Method

The hash function used for the multiplication method

$$\text{is } h(k) = \text{floor}(n(kA \bmod 1))$$

Here, k is the key and A can be any constant value between 0 and 1. Both k and A are multiplied and their fractional part is separated. This is then multiplied with n to get the hash value. An example of the Multiplication Method is as follows -

$$k=123$$

$$3$$

$$n=10$$

$$0$$

$$A=0.6$$

$$1803$$

$$3$$

$$h(123) = 100 (123 * 0.618033 \bmod 1)$$

$$= 100 (76.018059 \bmod 1)$$

$$= 100 (0.018059)$$

$$= 1$$

The hash value obtained is 1

An advantage of the multiplication method is that it can work with any value of A , although some values are believed to be better than others.

Mid Square Method

The mid square method is a very good hash function. It involves squaring the value of the key and then extracting the middle r digits as the hash value. The value of r can be decided

according to the size of the hash table.

An example of the Mid Square Method is as follows -

Suppose the hash table has 100 memory locations. So $r=2$ because two digits are required to map the key to memory location.

1. MID-SQUARE METHOD:

Ex: Key = 56789

$SQ(\text{Key}) = 3224990521$ (3 digit needed)

ADDRESS = 990

2. DIVISION METHOD:

$$H[x] = x \bmod M$$

$M \rightarrow$ To be an even number.

3. DIGITAL ANALYSIS METHOD:

Selection digits and reversing their
Order. (Eg): 7546123, selecting 3 to 6 position

Key = 2164

$$k = 50$$

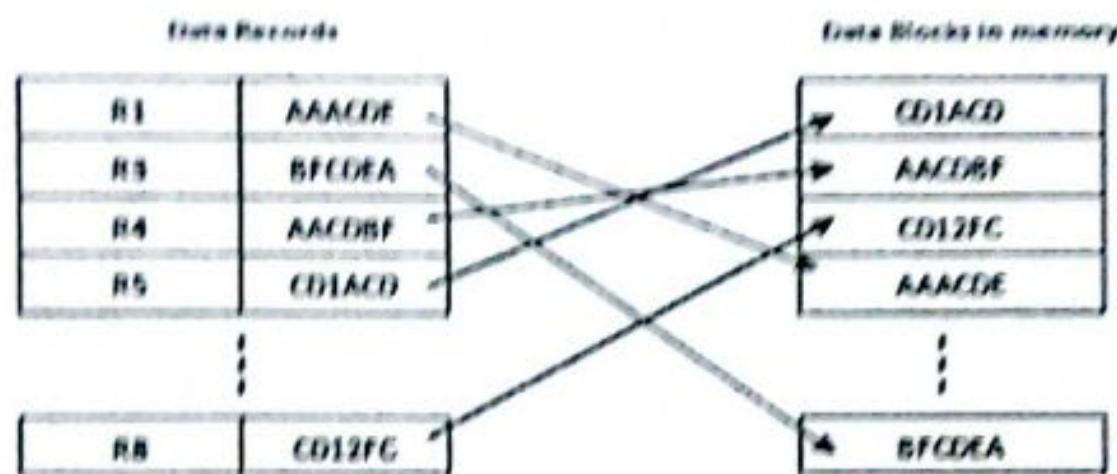
$$k * k = 2500$$

$$h(50) = 50$$

The hash value obtained is 50

2. List out the properties of an indexed sequential file and explain (Dec 2014)

This is an advanced sequential file organization method. Here records are stored in order of primary key in the file. Using the primary key, the records are sorted. For each primary key, an index value is generated and mapped with the record. This index is nothing but the address of record in the file.



In this method, if any record has to be retrieved, based on its index value, the data block address is fetched and the record is retrieved from memory.

Advantages of ISAM

- Since each record has its data block address, searching for a record in larger database is easy and quick. There is no extra effort to search records. But proper primary key has to be selected to make ISAM efficient.
- This method gives flexibility of using any column as key field and index will be generated based on that. In addition to the primary key and its index, we can have index generated for other fields too. Hence searching becomes more efficient, if there is search based on columns other than primary key.
- It supports range retrieval, partial retrieval of records. Since the index is based on the key value, we can retrieve the data for the given range of values. In the same way, when a

partial key value is provided, say student names starting with 'JA' can also be searched easily.

Disadvantages of ISAM

- An extra cost to maintain index has to be afforded. i.e.; we need to have extra space in the disk to store this index value. When there is multiple key-index combinations, the disk space will also increase.

As the new records are inserted, these files have to be restructured to maintain the sequence. Similarly, when the record is deleted, the space used by it needs to be released. Else, the performance of the database will slow down.

3. Explain static tree tables.(Nov 2015)

Static tree table - identifiers are known in advance and no. deletions or insertions are allowed.

STATIC TREE TABLE

Symbol tables with property that, identifiers are known in advance and no additions or deletions are performed are called static.

One solution is to sort the names and store them sequentially using binary search tree. The implementation of static tree table is carried out by binary search tree as follows,

BINARY SEARCH TREE

A binary search tree T is a binary tree, either it is empty or each node in the tree contains an identifier and

The 3 conditions for a tree to be a binary search tree are

All identifier in the left sub-tree of T are less (numerically and alphabetically) than the identifier in the root node

All identifier in the right sub-tree of T are greater (numerically and

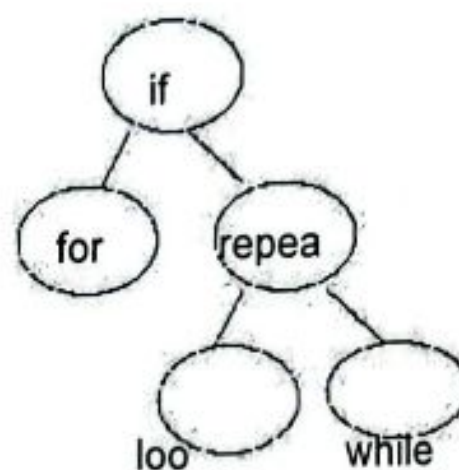
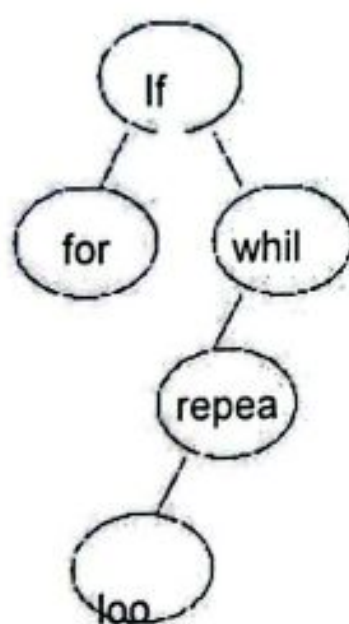
DS

alphabetically) than the identifier in the root node T

□ The left and right sub-trees of T are also binary search trees.

Example for a BST:

The following fig. shows two possible binary search trees for a sub set of reserved words.



SEARCH FOR A KEY IN BST:

To determine whether an identifier X is present in a BST, X is compared with the root.

If X is less than the identifier in the root, then the search continues in the root, the search terminates successfully, otherwise the search continues in the right sub tree.

ALGORITHM Procedure SEARCH (T,X,i)

// search the bst T for X. each node has fields LCHILD, IDENT, RCHILD. Return I = 0 if X is not in T. Otherwise, return I such that IDENT (i) = X. //

1. i ← T

2. while i ≠ 0 do

3. case
4. : X < IDENT(i) : I <- LCHILD(i) //search left sub-tree//
5. : X = IDENT(i) : return
6. : X > IDENT(i) : I <- RCHILD(i) // search right child//
7. end
8. end
9. end SEARCH

NODES IN BST:

here are 2 types of nodes available

- in BST Internal node
- External node

INTERNAL NODE: The node which has its own subnodes(child)

EXTERNAL NODE: The node which does not have subnodes and remains as terminal

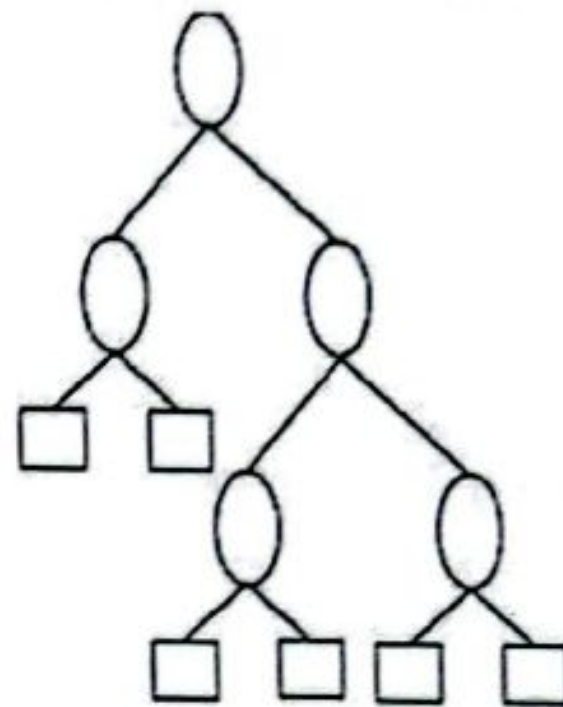
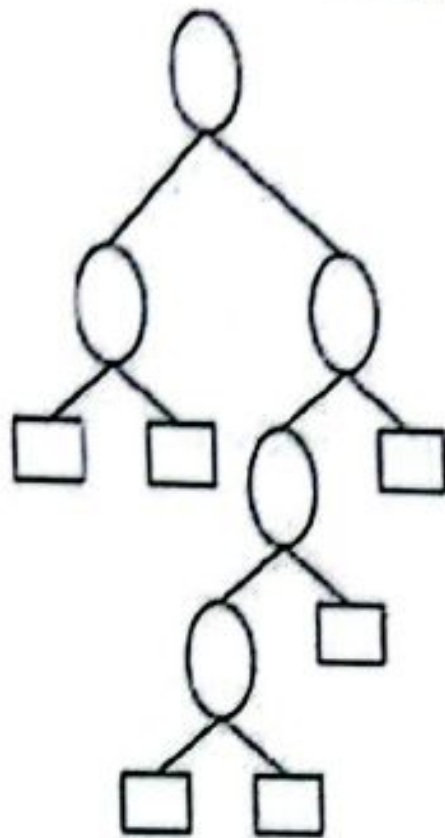
PATH LENGTH:

- The path length of BST is the sum of distances of the initial parent node to the last terminal of the tree
- There are two types of path length in BST
 - ,they are Internal Path Length
 - External Path Length

The calculation of the path lengths are as follows,

EXTERNAL PATH LENGTH:

DEFINITION: The external path length of a binary tree is the sum of the lengths of the path from the root to all external nodes.
For example consider the following diagram and its path length is calculated..



The external path length for the above figures are, FIGURE I:
 $E = 2 + 2 + 4 + 4 + 3 + 2 = 17$

FIGURE II: $E = 2 + 2 + 4 + 4 + 4 + 4 = 16$ Where, E is the external path length

INTERNAL PATH LENGTH

DEFINITION: The internal path length of a binary tree is the sum of the length of the path from the root node to all internal nodes.

The calculation of the internal path

length is as follows The internal path

length of the above diagram is

FIGUR

E I:

$$I=0+1$$

$$+1+2$$

$$+3=7$$

FIGU

RE II:

$$I=0+1$$

$$+1+2$$

$$+2=6$$

RELATION B/W INTERNAL &EXTERNAL PATH LENGTH

The relation b/w the internal and external path length is given by the equation $E=I+2n$

Where, E is the External path length I is the internal path length n is the no of internal nodes

In the below

picture,

$$E=2+2+2+3+4+$$

$$4=17$$

$$I=0+1+1+2+3=$$

$$7$$

Number of

internal nodes

$n=5$ Thus it

satisfies the

eqn. $E=I+2n$

WEIGHTED EXTERNAL PATH LENGTH:

- The weighted path length is calculated for the nodes which has its own magnitude
- The sum of the product of height of the node and its corresponding magnitude of all the external nodes of a tree is called as the external path length.
- The weighted external path length of a binary tree is defined to be
- $\sum (1 \leq i \leq n+1) [q_i k_i]$ where k_i is the distance from the root node to the external node with weight q_i . Consider the following diagram

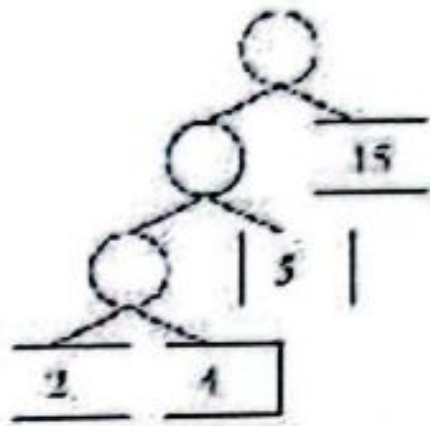
Their respective weighted external path lengths are,

$$\rightarrow 2.3 + 4.3 + 5.2 + 15.1 =$$

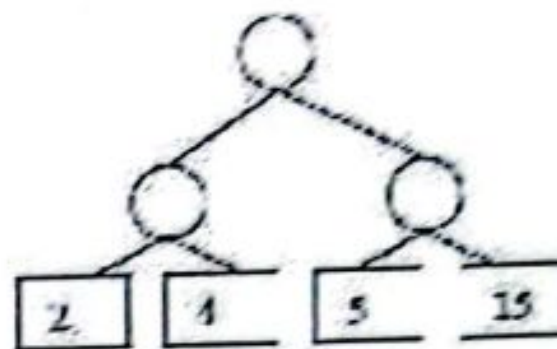
$$43 -$$

$$> 2.2 + 4.2 + 5.2 + 15.2 =$$

$$52$$



and



4. Explain the following in the design of hashing (a). Division Method, (b). Folding Method (Nov 2017)

(I) DIVISION METHOD

In this method, integer x is to divide by M and d then to use the remainder modulo M . The hash function is $H(x) = x \bmod M$

Great care should be taken while choosing value for M and preferably it should be an even number. By making M a large prime number the keys are spread out evenly.

(ii) FOLDING METHOD

A key is partitioned into a number of parts, each of which has the same length as the required address. The parts are then added together, ignoring the final carry, to form an address. For eg., if the key 356942781 is to be transformed into a three-digit address.

Two types:

1. Fold-shifting: 356, 942 and 781 are added to yield 079.
2. Fold-boundary method: 653, 942 and 187 are added together, yielding 782.

7. Explain the following in the design of hashing (a). Sequential organization (b). Indexing technique (Nov 2017).

We know that data is stored in the form of records. Every record has a key field, which helps it to be recognized uniquely.

Indexing is a data structure technique to efficiently retrieve records from the database files based on some attributes on which the indexing has been done. Indexing in database systems is similar to what we see in books.

Indexing is defined based on its indexing attributes. Indexing can be of the following types –

Primary Index – Primary Index is defined on an ordered data file. The data file is

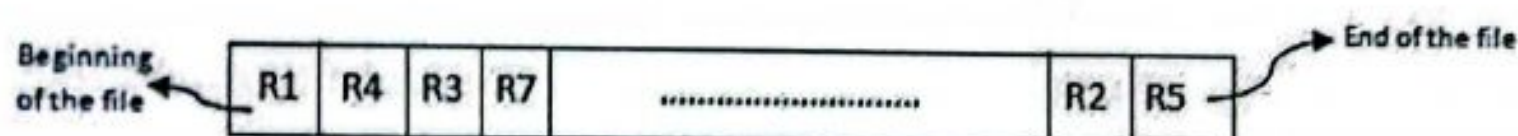
ordered on a key field. The key field is generally the primary key of the relation.

Secondary Index – Secondary index may be generated from a field which is a candidate key and has a unique value in every record, or a non-key with duplicate values.

5. Explain sequential file organization / Explain in detail about indexing techniques (Nov 12, Nov 13,14) FILE ORGANIZATION

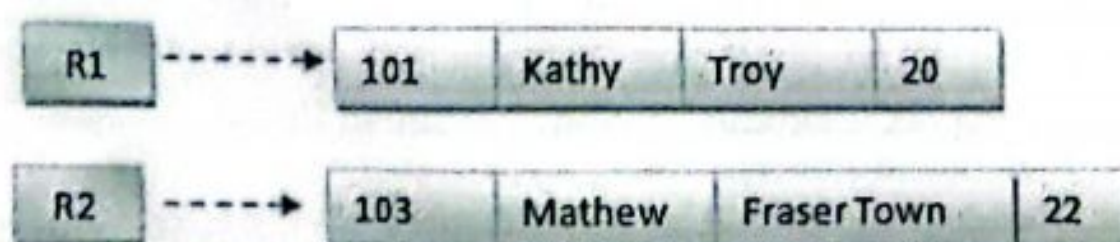
It is one of the simple methods of file organization. Here each file/records are stored one after the other in a sequential manner. This can be achieved in two ways:

Records are stored one after the other as they are inserted into the tables. This method is called **pile file method**. When a new record is inserted, it is placed at the end of the file. In the case of any modification or deletion of record, the record will be searched in the memory blocks. Once it is found, it will be marked for deleting and new block of record is entered.

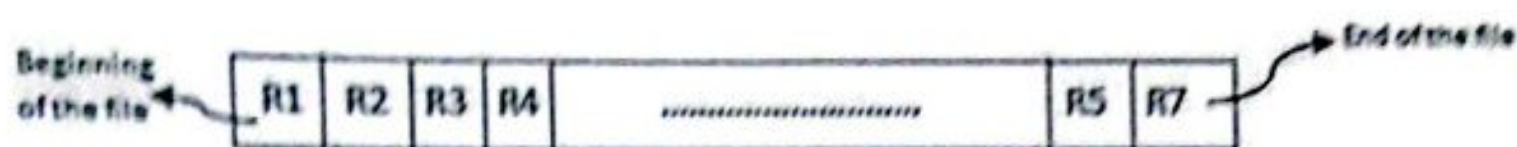


Inserting a new record:

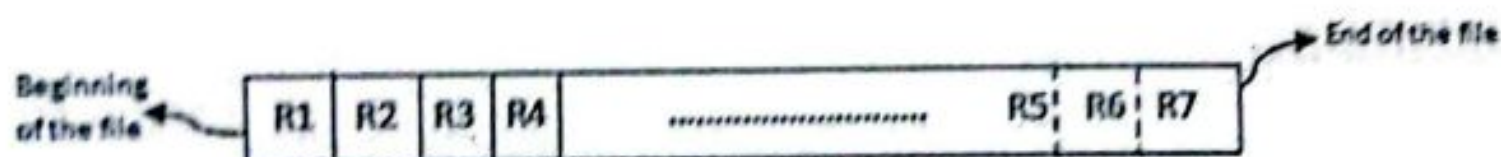
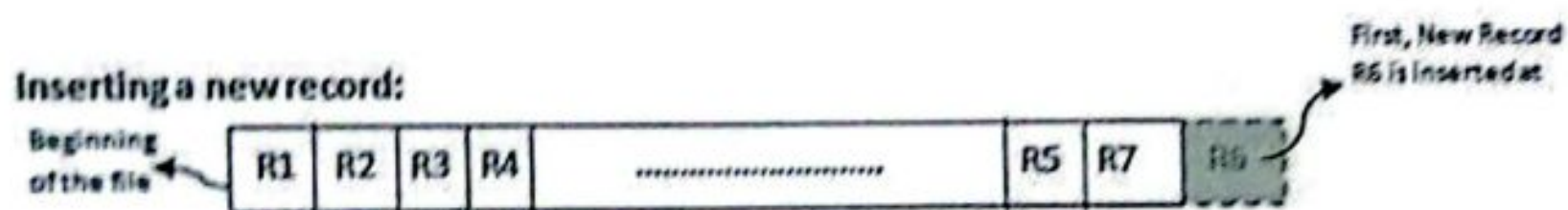
In the diagram above, R1, R2, R3 etc are the records. They contain all the attribute of a row. i.e.; when we say student record, it will have his id, name, address, course, DOB etc. Similarly R1, R2, R3 etc can be considered as one full set of attributes.



In the second method, records are sorted (either ascending or descending) each time they are inserted into the system. This method is called **sorted file method**. Sorting of records may be based on the primary key or on any other columns. Whenever a new record is inserted, it will be inserted at the end of the file and then it will sort – ascending or descending based on key value and placed at the correct position. In the case of update, it will update the record and then sort the file to place the updated record in the right place. Same is the case with delete.



Inserting a new record:



Advantages of Sequential File Organization

- The design is very simple compared other file organization.
There is no much effort involved to store the data.
- When there are large volumes of data, this method is very fast and efficient. This method is helpful when most of the records have to be accessed like calculating the grade of a student, generating the salary slips etc where we use all the records for our calculations

- This method is good in case of report generation or statistical calculations.
- These files can be stored in magnetic tapes which are comparatively cheap

INDEXED SEQUENTIAL FILE

Each record of a file has a key field which uniquely identifies that record. An index consists of keys and address (physical disc locations)

An index sequential file is a sequential file (i.e. sorted into order of a key field) which has an index.

A full index to a file is one in which there is an entry for every record.

Indexed sequential file are important for applications where data needs to be accessed sequentially and randomly using the index.

An indexed sequential file allows fast access to a specific record.

E.g.:- A company may store details about its employees as an indexed sequential file. Sometimes the file is accessed,

Sequentially, for e.g.:- When the whole of the file is processed to produce pay slips at the end of the month.

Randomly, may be an employee changes address, or a female employee gets married and changes her surname.

Disadvantage of Sequential Files - The retrieval of a record from a sequential file, on average, requires access to half the records in the file, making such enquiries not only inefficient but very time consuming for large files. To improve the query responses time of a sequential file, a type of indexing technique can be added.

Definition - An index is a set of y , address pairs. A sequential (or sorted on primary keys) file that is indexed is called an index sequential file. The index provides for random access to records, while the sequential nature of the file provides easy

access to the subsequent records as well as sequential processing.

STRUCTURE OF INDEX SEQUENTIAL FILES

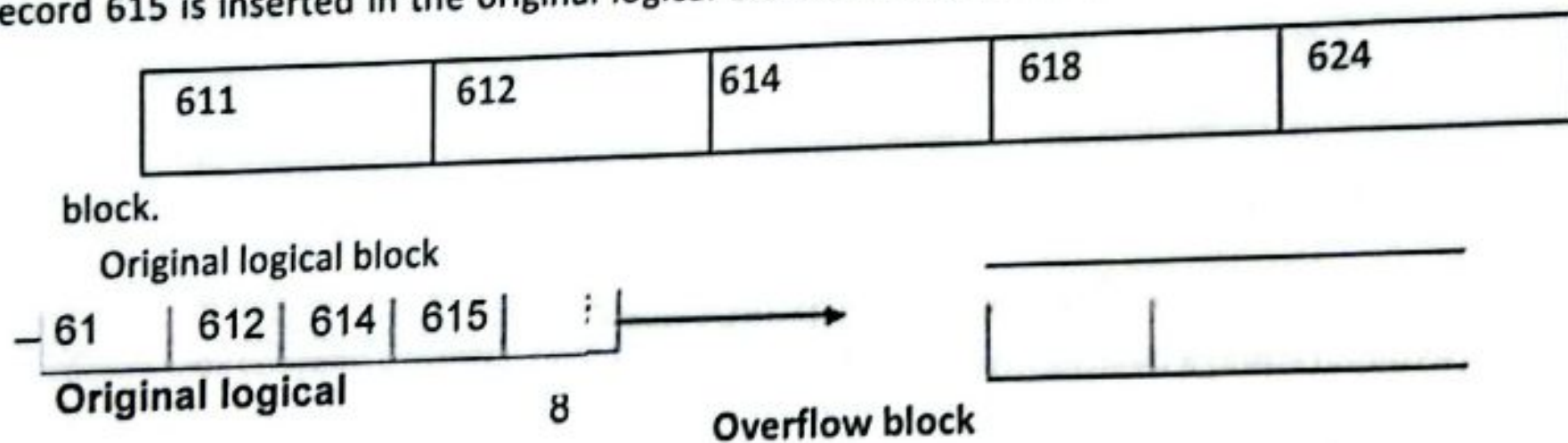
An index-sequential file consists of the data plus one or more levels of indexes.

When inserting a record, we have to maintain the sequence of records and this may necessitate shifting subsequent records.

For a large file this is a costly and inefficient process, instead, the records that overflow then logical area is shifted into a designated overflow area and a pointer is provided in the logical area of associated index entry points to the overflow location.

This is illustrated below (figure).

Record 615 is inserted in the original logical block causing a record to be moved to an overflow



When records are forced into the overflow area as a result of insertion, the insertion process is simplified, but the search time is increased.

Deletion of records from index-sequential files creates logical gaps:-

- The records are not physically removed but only flagged as having been deleted.
- If there were a number of deletions, we may have a great amount of unused space.

6. Write short note on (a).Inverted tables.(b).Static tree table.(May 2018).

In many cases, a table holding a collection of records must support efficient lookup by more than one key value. For example, we may have a set of customer records consisting of a name field, an address field and a phone number field. If we use a simple array, sorted by name, we have good support for searching on the name field, but not for address or phone number searches:

An inverted index is an index data structure storing a mapping from content, such as words or numbers, to its locations in a document or a set of documents. In simple words, it is a hashmap like data structure that directs you from a word to a document or a web page.

There are two types of inverted indexes: A record-level inverted index contains a list of references to documents for each word. A word-level inverted index additionally contains the positions of each word within a document. The latter form offers more functionality, but needs more processing power and space to be created.

7. Explain the two file organization methods. Explain how is sorting done in external storage devices.(May 2018,Nov 2019,May 2017).

There are three types of organizing the file:

1. Sequential access file organization
2. Direct access file organization
3. Indexed sequential access file organization

1. Sequential access file organization:

- Storing and sorting in contiguous block within files on tape or disk is called as sequential access file organization.
- In sequential access file organization, all records are stored in a sequential order. The records are arranged in the ascending or descending order of a key field.

- Sequential file search starts from the beginning of the file and the records can be added at the end of the file.
- In sequential file, it is not possible to add a record in the middle of the file without rewriting the file.

Advantages of sequential file

- It is simple to program and easy to design.
- Sequential file is best use if storage space.

Disadvantages of sequential file

- Sequential file is time consuming process.
- It has high data redundancy.
- Random searching is not possible.

2. Direct access file organization

- Direct access file is also known as random access or relative file organization.
- In direct access file, all records are stored in direct access storage device (DASD), such as hard disk. The records are randomly placed throughout the file.
- The records does not need to be in sequence because they are updated directly and rewritten back in the same location.
- This file organization is useful for immediate access to large amount of information. It is used in accessing large databases.
- It is also called as hashing.

Advantages of direct access file organization

- Direct access file helps in online transaction processing system (OLTP) like online railway reservation system.
- In direct access file, sorting of the records are not required.
- It accesses the desired records immediately.
- It updates several files quickly.
- It has better control over record allocation.

Disadvantages of direct access file organization

Direct access file does not provide back up facility.

- It is expensive.
- It has less storage space as compared to sequential file.

3. Indexed sequential access file organization

- Indexed sequential access file combines both sequential file and direct access file organization.
- In indexed sequential access file, records are stored randomly on a direct access device such as magnetic disk by a primary key.
- This file have multiple keys. These keys can be alphanumeric in which the records are ordered is called primary key.
- The data can be access either sequentially or randomly using the index. The index is stored in a file and read into memory when the file is opened.

Advantages of Indexed sequential access file organization

- In indexed sequential access file, sequential file and random file access is possible.
- It accesses the records very fast if the index table is properly organized.
- The records can be inserted in the middle of the file.
- It provides quick access for sequential and direct processing.
- It reduces the degree of the sequential search.

Disadvantages of Indexed sequential access file organization

Indexed sequential access file requires unique keys and periodic reorganization.

- Indexed sequential access file takes longer time to search the index for the data access or retrieval.
- It requires more storage space.
- It is expensive because it requires special software.
- It is less efficient in the use of storage space as compared to other file organizations.

8. Explain Hash Indexing with an example.(Nov 2019).

What Is Hashing?

- Hashing is the process of mapping large amount of data item to smaller table with the help of hashing function.
- Hashing is also known as Hashing Algorithm or Message Digest Function.
- It is a technique to convert a range of key values into a range of indexes of an array.
- It is used to facilitate the next level searching method when compared with the linear or binary search.
- Hashing allows to update and retrieve any data entry in a constant time $O(1)$.
- Constant time $O(1)$ means the operation does not depend on the size of the data.
- Hashing is used with a database to enable items to be retrieved more quickly.
- It is used in the encryption and decryption of digital signatures.

What Is Hash Function?

- A fixed process converts a key to a hash key is known as a Hash Function.
- This function takes a key and maps it to a value of a certain length which is called a Hash value or Hash.
- Hash value represents the original string of characters, but it is normally smaller than the original.
- It transfers the digital signature and then both hash value and signature are sent to the receiver. Receiver uses the same hash function to generate the hash value and then compares it to that received with the message.
- If the hash values are same, the message is transmitted without errors.

What Is Hash Table?

- Hash table or hash map is a data structure used to store key-value pairs.
- It is a collection of items stored to make it easy to find them later.
- It uses a hash function to compute an index into an array of buckets or slots from which the desired value can be found.

- It is an array of list where each list is known as bucket.
- It contains value based on the key.
- Hash table is used to implement the map interface and extends Dictionary class.
- Hash table is synchronized and contains only unique elements.

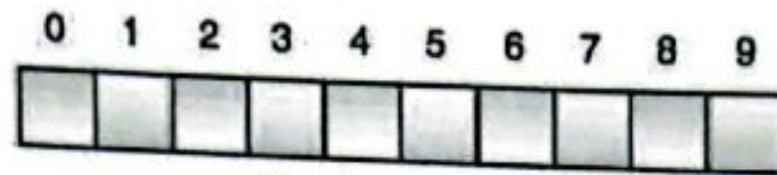


Fig. Hash Table

- The above figure shows the hash table with the size of $n = 10$. Each position of the hash table is called as **Slot**. In the above hash table, there are n slots in the table, names = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}. Slot 0, slot 1, slot 2 and so on. Hash table contains no items, so every slot is empty.
- As we know the mapping between an item and the slot where item belongs in the hash table is called the hash function. The hash function takes any item in the collection and returns an integer in the range of slot names between 0 to $n-1$.
- Suppose we have integer items {26, 70, 18, 31, 54, 93}. One common method of determining a hash key is the division method of hashing and the formula is :

$$\text{Hash Key} = \text{Key Value} \% \text{Number of Slots in the Table}$$

- Division method or reminder method takes an item and divides it by the table size and returns the remainder as its hash value.

Data Item	Value % No. of Slots	ash Value
26	$26 \% 10 = 6$	6
70	$70 \% 10 = 0$	0
18	$18 \% 10 = 8$	8
31	$31 \% 10 = 1$	1
54	$54 \% 10 = 4$	4
93	$93 \% 10 = 3$	3

0	1	2	3	4	5	6	7	8	9
70	31		93	54		26		18	

Fig. Hash Table

- After computing the hash values, we can insert each item into the hash table at the designated position as shown in the above figure. In the hash table, 6 of the 10 slots are occupied, it is referred to as the load factor and denoted by, $\lambda = \text{No. of items} / \text{table size}$. For example, $\lambda = 6/10$.
- It is easy to search for an item using hash function where it computes the slot name for the item and then checks the hash table to see if it is present.
- Constant amount of time $O(1)$ is required to compute the hash value and index of the hash table at that location.

Linear Probing

- Take the above example, if we insert next item 40 in our collection, it would have a hash value of 0 ($40 \% 10 = 0$). But 70 also had a hash value of 0, it becomes a problem. This problem is called as **Collision** or **Clash**. Collision creates a problem for hashing technique.
- Linear probing is used for resolving the collisions in hash table, data structures for maintaining a collection of key-value pairs.
- Linear probing was invented by Gene Amdahl, Elaine M. McGraw and Arthur Samuel in 1954 and analyzed by Donald Knuth in 1963.
- It is a component of open addressing scheme for using a hash table to solve the dictionary problem.
- The simplest method is called Linear Probing. Formula to compute linear probing is:

$$P = (1 + P) \% (\text{MOD})$$

It takes too much time to find an empty slot.

Quadratic probing

In this, when the collision occurs, we probe for i^2 th slot in i th iteration, and this probing is performed until an empty slot is found. The cache performance in quadratic probing is lower than the linear probing. Quadratic probing also reduces the problem of clustering.

Double hashing

In this, you use another hash function, and probe for $(i * \text{hash}_2(x))$ in the i th iteration. It takes longer to determine two hash functions. The double probing gives the very poor the cache performance, but there has no clustering problem in it.